

Task 05 — Technical Explanation

Apex Integration with ExchangeRate-API

1. Secure API Connection and Key Management

The ExchangeRate-API key is never stored in Apex code, never hardcoded into a URL, and never appears in any debug log. It lives in exactly one place: a Custom Header on a Salesforce External Credential, set to "Authorization: Bearer <key>". A Named Credential (URL: <https://v6.exchangerate-api.com/v6>) is linked to that External Credential and used as the callout endpoint (callout:ExchangeRateAPI/...). Salesforce injects the Authorization header automatically on every callout made through the Named Credential — the Apex code that makes the callout never sees or references the key at all. Access to the credential is scoped through a dedicated Permission Set ("Currency Integration Access") granting External Credential Principal Access, rather than being open to every user by default.

2. Opportunity Grouping and Processing Efficiency

Before any callout is made, every Opportunity in the batch is grouped by its Expected Currency Type into a Map<String, List<Opportunity>>. The API is then called exactly once per distinct currency present in that group — not once per Opportunity. For a 200-record chunk containing, for example, 40 USD, 35 EUR, and 30 JPY records, this means 3 callouts total instead of 105.

This grouping is done inside CurrencyConversionService.processOpportunities(), which is the single shared entry point used by both the bulk batch process and the real-time trigger automation — so the efficiency logic is never duplicated or at risk of drifting between the two paths.

3. Avoiding Unnecessary API Requests

Two further checks happen before any callout for a given currency:

- A currency is only queried if it appears in the supported list (USD, EUR, GBP, JPY, AUD, CAD), as specified in the assignment. A currency outside this list is marked Failed immediately, with no API call made for it at all.
- Opportunities with a blank Expected Currency Type are skipped entirely and never enter the grouping map, so they never contribute to a callout.

The reprocessing feature (Section 5 below) extends this principle further: a correction run can be scoped to only Failed or Pending records via a query filter, so records that already converted successfully are never re-queried unnecessarily.

4. Currency and Opportunity Update Tracking

Two custom objects provide a complete, queryable audit trail, kept deliberately separate because they track different things:

Exchange_Rate_History__c — one record per currency, per processing run

Captures Base Currency, Target Currency, Exchange Rate, Retrieved Date/Time, API Status (Success/Failed), and Error Details. This is the rate-level audit trail: it answers "what rate did we get for EUR, and when, and did the call succeed."

Opportunity_Currency_Update__c — one record per successful conversion

Captures the Opportunity lookup, the original BDT Amount, the target currency, the rate used, and both the Previous and New converted amounts. This is the record-level audit trail: it answers "what did this specific Opportunity's converted amount change from and to, and using what rate."

On the Opportunity itself, six fields (Expected_Currency_Type__c, Expected_Currency_Amount__c, Exchange_Rate__c, Currency_Last_Updated__c, Currency_Update_Status__c, Currency_Error_Message__c) hold the current, at-a-glance state.

5. Failure Handling and Reprocessing

Every Opportunity is evaluated against a fixed order of checks: blank Amount fails first, then an unsupported currency code fails, then an API-level failure fails, and only then is a successful conversion calculated. In every failure case, the existing Expected_Currency_Amount__c value is left completely untouched — only the Status and Error Message fields change, so a temporary API outage or an unsupported code never destroys previously good data.

Database.update(oppsToUpdate, false) and Database.insert(list, false) are used throughout (rather than plain insert/update), so that a single problematic record never rolls back or blocks the processing of every other record in the same run — each record succeeds or fails independently.

Reprocessing is supported natively: CurrencyConversionBatch accepts an optional query filter in its constructor. Calling it with no argument processes every Opportunity; calling it with a filter such as "Currency_Update_Status__c IN ('Failed', 'Pending')" reprocesses only those records, leaving already-successful conversions untouched. This was used in practice during development to correct a batch of records after a logic fix, without disturbing the rest of the dataset.

```
// Full run against every Opportunity
Database.executeBatch(new CurrencyConversionBatch(), 200);

// Reprocess only records that are Failed or still Pending
Database.executeBatch(
    new CurrencyConversionBatch('Currency_Update_Status__c IN (\'Failed\', \'Pending\')'), 200);
```

6. Testing of External Callouts

No test in the suite makes a real HTTP request. Every test that exercises callout-dependent code implements HttpCalloutMock and registers it with Test.setMock(HttpCalloutMock.class, ...) before the callout-triggering action, per Salesforce's standard mocking pattern. Mock classes return canned JSON matching the real API's actual response shapes for both success ({"result": "success", "conversion_rate": ...}) and documented failure cases (invalid-key, unsupported-code, and a deliberately malformed response to test JSON-parsing resilience).

12 test methods across 4 test classes (ExchangeRateApiClientTest, CurrencyConversionServiceTest, CurrencyConversionBatchTest, OpportunityCurrencyTriggerTest) cover: a successful conversion, every documented API error type, blank-input short-circuiting (proven by making zero callouts), bulk batch processing, the filtered reprocess path, and every trigger scenario (insert, a qualifying update, a non-qualifying update, and the bypass flag used during bulk data loads). All 12 pass together in a single run, with 85–100% coverage per class.

7. Field-Level Security and Access Enforcement

CurrencyConversionService.processOpportunities() begins with an explicit enforceFieldLevelSecurity() check before touching any data. It verifies, using Schema describe calls, that the running user can read the Opportunity fields the logic depends on (Amount, Expected_Currency_Type__c, Expected_Currency_Amount__c), can update the six currency-tracking fields on Opportunity, and can create records on both Exchange_Rate_History__c and Opportunity_Currency_Update__c. If any of these checks fail, the method throws a custom InsufficientAccessException naming the missing permission and pointing to the "Currency Integration Access" Permission Set, rather than silently skipping records or partially updating data.

This check is deliberately fail-loud rather than fail-silent: an automated integration that quietly does nothing when misconfigured is far harder to diagnose than one that raises a clear exception the first time it runs. All classes are also declared "with sharing", so record-level sharing rules are respected in addition to this field-level check.